

459. Repeated Substring Pattern

Easy Topics Companies

Given a string `s`, check if it can be constructed by taking a substring of it and appending multiple copies of the substring together.

Example 1:

Input: `s = "abab"`
Output: `true`
Explanation: It is the substring "ab" twice.

Example 2:

Input: `s = "aba"`
Output: `false`

Example 3:

Input: `s = "abcabcabcabc"`
Output: `true`
Explanation: It is the substring "abc" four times or the substring "abcabc" twice.

Constraints:

- $1 \leq s.length \leq 10^4$
- `s` consists of lowercase English letters.



Seen this question in a real interview before? 1/6

Yes No

```
14         for (int i = len; i < n; i++) {
15             if (s.charAt(i) != s.charAt(i % len)) {
16                 valid = false;
17                 break;
18             }
19         }
20
21         if (valid) {
22             return true;
23         }
24     }
25     return false;
26 }
27
28 }
```

Saved

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

`s =`
`"abab"`

Output

`true`

Expected

`true`

Given two strings, determine if they share a common substring. A substring may be as small as one character.

Example

$s1 = \text{'and'}$

$s2 = \text{'art'}$

These share the common substring *a*.

$s1 = \text{'be'}$

$s2 = \text{'cat'}$

These do not share a substring.

Function Description

Complete the function `twoStrings` in the editor below.

`twoStrings` has the following parameter(s):

- string `s1`: a string
- string `s2`: another string

Returns

- string: either YES or NO

Input Format

The first line contains a single integer *p*, the number of test cases.

The following *p* pairs of lines are as follows:

- The first line contains string *s1*.
- The second line contains string *s2*.

```
import java.io.*;

public class Solution {

    public static String twoStrings(String s1, String s2) {
        boolean[] present = new boolean[26];

        // Store characters of s1
        for (int i = 0; i < s1.length(); i++) {
            present[s1.charAt(i) - 'a'] = true;
        }

        // Check characters of s2
        for (int i = 0; i < s2.length(); i++) {
            if (present[s2.charAt(i) - 'a']) {
                return "YES";
            }
        }

        return "NO";
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br =
            new BufferedReader(new InputStreamReader(System.in));

        // ...
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ **Sample Test case 0**

✔ Sample Test case 1

✔ Sample Test case 2

Input (stdin)

```
1 2
2 hello
3 world
4 hi
5 world
```

Your Output (stdout)

```
1 YES
2 NO
```

Expected Output

For two strings A and B, we define the similarity of the strings to be the length of the longest prefix common to both strings. For example, the similarity of strings "abc" and "abd" is 2, while the similarity of strings "aaa" and "aaab" is 3.

Calculate the sum of similarities of a string S with each of its suffixes.

Input Format

The first line contains the number of test cases t .

Each of the next t lines contains a string to process, s .

Constraints

- $1 \leq t \leq 10$
- $1 \leq |s| \leq 100000$
- s is composed of characters in the range `ascii[a-z]`

Output Format

Output t lines, each containing the answer for the corresponding test case.

Sample Input

```
2
ababaa
aa
```

Sample Output

```
11
2
```

```
import java.io.*;
```

```
public class Solution {
```

```
    public static long stringSimilarity(String s) {
```

```
        int n = s.length();
```

```
        int[] z = new int[n];
```

```
        int left = 0;
```

```
        int right = 0;
```

```
        for (int i = 1; i < n; i++) {
```

```
            if (i <= right) {
```

```
                z[i] = Math.min(right - i + 1, z[i - left]);
```

```
            }
```

```
            while (i + z[i] < n &&
```

```
                    s.charAt(z[i]) == s.charAt(i + z[i])) {
```

```
                z[i]++;
```

```
            }
```

```
            if (i + z[i] - 1 > right) {
```

```
                left = i;
```

```
                right = i + z[i] - 1;
```

```
            }
```

```
        }
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

✔ Sample Test case 1

Input (stdin)

1	2
2	ababaa
3	aa

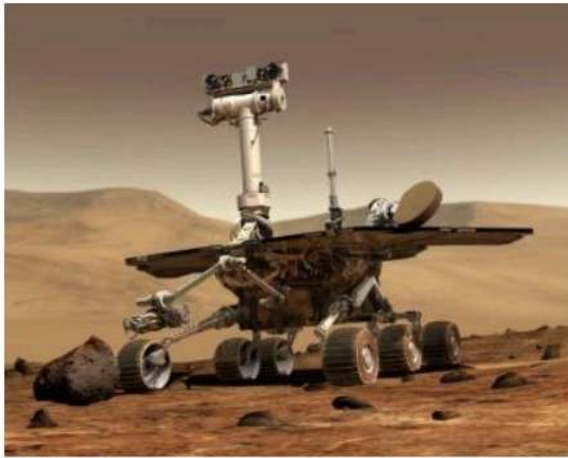
Your Output (stdout)

1	11
2	3

Expected Output

1	11
2	3

A space explorer's ship crashed on Mars! They send a series of SOS messages to Earth for help.



Letters in some of the SOS messages are altered by cosmic radiation during transmission. Given the signal received by Earth as a string, s , determine how many letters of the SOS message have been changed by radiation.

Example

$s = \text{'SOSTOT'}$

The original message was SOS SOS. Two of the message's characters were changed in transit.

Function Description

Complete the marsExploration function in the editor below.

marsExploration has the following parameter(s):

- string s : the string as received on Earth


```
import java.io.*;

public class Solution {

    public static int marsExploration(String s) {
        int count = 0;
        String pattern = "SOS";

        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) != pattern.charAt(i % 3)) {
                count++;
            }
        }

        return count;
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br =
            new BufferedReader(new InputStreamReader(System.in));

        String s = br.readLine().trim();

        System.out.println(marsExploration(s));
    }
}
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
1 SOSSPSSQSSOR
```

Sample Test case 1

Sample Test case 2

Your Output (stdout)

```
1 3
```

Expected Output

```
1 3
```

3) WhatsApp

Rotate String - LeetCode

Contains Duplicate Solution

+

Ask Gemini

Verify it's you

leetcode.com/problems/rotate-string/

Problem List

Run

Ctrl

Submit

+

Description

Editorial

Solutions

Submissions

796. Rotate String

EasyTopicsCompanies

Given two strings `s` and `goal`, return `true` if and only if `s` can become `goal` after some number of **shifts** on `s`.

A **shift** on `s` consists of moving the leftmost character of `s` to the rightmost position.

- For example, if `s = "abcde"`, then it will be `"bcdea"` after one shift.

Example 1:

Input: `s = "abcde"`, `goal = "cdeab"`
Output: `true`

Example 2:

Input: `s = "abcde"`, `goal = "abcd"`
Output: `false`

Constraints:

- `1 <= s.length, goal.length <= 100`
- `s` and `goal` consist of lowercase English letters.

Discover moreFinding Math Tutors

Seen this question in a real interview before? 1/6

YesNo

Accepted 1,009,557/1.3M Acceptance Rate 66.6%

5.1K30289 Online

Code

Note

×

JavaAuto

```
1 class Solution {
2     public boolean rotateString(String s, String goal)
3     {
4         // Lengths must be equal
5         if (s.length() != goal.length()) {
6             return false;
7         }
8
9         // goal must be a substring of s + s
10        String doubled = s + s;
11
12        return doubled.contains(goal);
13    }
14 }
```

SavedLn 14, Col 6

Testcase>Test Result

AcceptedRuntime: 0 ms

Case 1Case 2

Input

s =

"m"

goal =

34°C

Mostly sunny

Search

ENGUS

11:57

28-08-2026

Problem List

Submit

00:07:29

Premium

Description

Editorial

Solutions

Submissions

438. Find All Anagrams in a String

MediumTopicsCompanies

Given two strings `s` and `p`, return an array of all the start indices of `p`'s **anagrams** in `s`. You may return the answer in **any order**.

Example 1:

Input: `s = "cbaebabacd", p = "abc"`

Output: `[0,6]`

Explanation:

The substring with start index = 0 is "cba", which is an anagram of "abc".

The substring with start index = 6 is "bac", which is an anagram of "abc".

Example 2:

Input: `s = "abab", p = "ab"`

Output: `[0,1,2]`

Explanation:

The substring with start index = 0 is "ab", which is an anagram of "ab".

The substring with start index = 1 is "ba", which is an anagram of "ab".

The substring with start index = 2 is "ab", which is an anagram of "ab".

Constraints:

13.4K187102 Online

Code

JavaAuto

```
1 import java.util.*;
2
3 class Solution {
4     public List<Integer> findAnagrams(String s, String p) {
5         List<Integer> result = new ArrayList<>();
6
7         if (s.length() < p.length()) {
8             return result;
9         }
10    }
```

SavedLn 48, Col 2

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2

Input

s =

"cbaebabacd"

p =

"abc"

Output

[0,6]

Given a string of lowercase letters in the range `ascii[a-z]`, determine the index of a character that can be removed to make the string a [palindrome](#). There may be more than one solution, but any will do. If the word is already a palindrome or there is no solution, return `-1`. Otherwise, return the index of a character to remove.

Example

`s = "bcbc"`

Either remove 'b' at index `0` or 'c' at index `3`.

Function Description

Complete the `palindromeIndex` function in the editor below.

`palindromeIndex` has the following parameter(s):

- string `s`: a string to analyze

Returns

- int: the index of the character to remove or `-1`

Input Format

The first line contains an integer q , the number of queries.

Each of the next q lines contains a query string s .

Constraints

- $1 \leq q \leq 20$
- $1 \leq \text{length of } s \leq 10^5 + 5$
- All characters are in the range `ascii[a-z]`.

Sample Input

```

public class Solution {
    public static int palindromeIndex(String s) {
        // Try removing left character
        if (isPalindrome(s, left + 1, right)) {
            return left;
        }

        // Try removing right character
        if (isPalindrome(s, left, right - 1)) {
            return right;
        }

        return -1;
    }

    left++;
    right--;
}

// Already a palindrome
return -1;
}

public static boolean isPalindrome(String s, int left, int right) {
    while (left < right) {
        if (s.charAt(left) != s.charAt(right)) {
            return false;
        }
    }
}

```

Line: 61 Col: 2

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Compiler Message

Success

Input (stdin)

13

2aaab

3baa

4aaa

Your Output (stdout)

13

20

Download

Find the Index of the First Occurrence in a String

Rotate String - LeetCode

Contains Duplicate Solution

Ask Gemini

Verify it's you

leetcode.com/problems/find-the-index-of-the-first-occurrence-in-a-string/

Problem List

Run Ctrl

Submit

Description

Editorial

Solutions

Submissions

28. Find the Index of the First Occurrence in a String

EasyTopicsCompanies

Given two strings `needle` and `haystack`, return the index of the first occurrence of `needle` in `haystack`, or `-1` if `needle` is not part of `haystack`.

Example 1:

Input: `haystack = "sadbutsad", needle = "sad"`
Output: `0`
Explanation: "sad" occurs at index 0 and 6. The first occurrence is at index 0, so we return 0.

Example 2:

Input: `haystack = "leetcode", needle = "leeto"`
Output: `-1`
Explanation: "leeto" did not occur in "leetcode", so we return -1.

Constraints:

- `1 <= haystack.length, needle.length <= 10^4`
- `haystack` and `needle` consist of only lowercase English characters.

class Solution {
 public int strStr(String haystack, String needle)
 {
 int n = haystack.length();
 int m = needle.length();
 for (int i = 0; i <= n - m; i++) {
 int j = 0;
 while (j < m && haystack.charAt(i + j) == needle.charAt(j)) {
 j++;
 }
 if (j == m) {
 return i;
 }
 }
 return -1;
 }
}

SavedLn 22, Col 3

Testcase>Test Result

AcceptedRuntime: 0 ms

Case 1Case 2

Input

haystack =
"sadbutsad"

needle =

Seen this question in a real interview before? 1/6

YesNo

Accepted 4,319,763/9.1MAcceptance Rate 47.2%

7.8K578165 Online

bluehostBuild apps on your VPS

34°C Mostly sunny

Search

ENG US12:0028-08-2026

5. Longest Palindromic Substring

Medium Topics Companies Hint

Given a string `s`, return the longest *palindromic substring* in `s`.

Example 1:
Input: `s = "babad"`
Output: `"bab"`
Explanation: `"aba"` is also a valid answer.

Example 2:
Input: `s = "cbbd"`
Output: `"bb"`

- Constraints:**
- `1 <= s.length <= 1000`
 - `s` consist of only digits and English letters.

 Gemini Notebook

Transform your study notes

Learn More

Seen this question in a real interview before? 1/6

Yes No

Accepted 5,041,550/13.1M Acceptance Rate 38.5%

Topics

```
25         return s.substring(start, end + 1);
26     }
27
28     private int expandAroundCenter(String s, int left, int right) {
29
30         while (left >= 0 && right < s.length()
31               && s.charAt(left) == s.charAt(right)) {
32             left--;
33             right++;
34         }
35         return right - left - 1;
36     }
37
38 }
```

Saved

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`s =`
`"babad"`

Output

`"bab"`

Expected

`"bab"`

A palindrome is a string that reads the same from left to right as it does from right to left.

Given a string, S , of N lowercase English letters, we define a k -length rotation as cutting the first k characters from the beginning of S and appending them to the end of S . For each S , there are N possible k -length rotations (where $0 \leq k < N$). See the Explanation section for examples.

Given N and S , find all N k -length rotations of S ; for each rotated string, S_k , print the maximum possible length of any palindromic substring of S_k on a new line.

Input Format

The first line contains an integer, N (the length of S).

The second line contains a single string, S .

Constraints

- $1 \leq N \leq 5 \times 10^5$
- $0 \leq k < N$
- S is comprised of lowercase English letters.

Output Format

There should be N lines of output, where each line k contains an integer denoting the maximum length of any palindromic substring of rotation S_k .

Sample Input 0

```
13
aaaaabbbbbaaaa
```

```

public class Solution {
    public static int longestPalindrome(String s) {

        for (int i = 0; i < n; i++) {
            t[2 * i + 2] = s.charAt(i);
            t[2 * i + 3] = '#';
        }

        t[2 * n + 2] = '$';

        int[] p = new int[t.length];
        int center = 0;
        int right = 0;
        int maxLength = 0;

        for (int i = 1; i < t.length - 1; i++) {

            int mirror = 2 * center - i;

            if (i < right) {
                p[i] = Math.min(right - i, p[mirror]);
            }

            while (t[i + (1 + p[i])] ==
                t[i - (1 + p[i])]) {
                p[i]++;
            }
        }
    }
}

```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0	Input (stdin) 113 2aaaaabbbbbaaaa
Sample Test case 1	Your Output (stdout) 112 212 310 48 58 69 711
Sample Test case 2	